

K-디지털 트레이닝

스프링 컨텍스트- 추상화

[KB] IT's Your Life

계약 정의를 위한 인터페이스 사용

✓ 인터페이스

- 특정 책임을 선언하는데 사용하는 추상 구조
- 인터페이스를 구현하는 객체는 이 책임을 정의해야 함
 - 동일한 인터페이스를 구현하는 여러 객체는 해당 인터페이스가 선언한 책임을 다른 방식으로 정의

✓ 인터페이스를 사용하여 계약을 정의

✓ 구현 분리를 위해 인터페이스 사용

○ 배송 앱에서 배송할 세부 정보를 인쇄하는 객체 구현

- 인쇄된 세부 정보는 목적지 주소별로 정렬되어 있어야 함
- 배송 주소지별로 패키지를 정렬하는 책임을 다른 객체에 위임해야 함

인쇄하기 전에 세부 정보를
정렬하려면 배송 주소별로
배송 세부 정보를 정렬해야 한다.



SorterByAddress 객체는
정렬 책임을 구현한다.



DeliveryDetailsPrinter는 정렬 책임을 다른 객체에 위임한다.

✓ 구현 분리를 위해 인터페이스 사용

- 배송 앱에서 배송할 세부 정보를 인쇄하는 객체 구현

```
public class DeliveryDetailsPrinter {
```

```
    private SorterByAddress sorter;
```

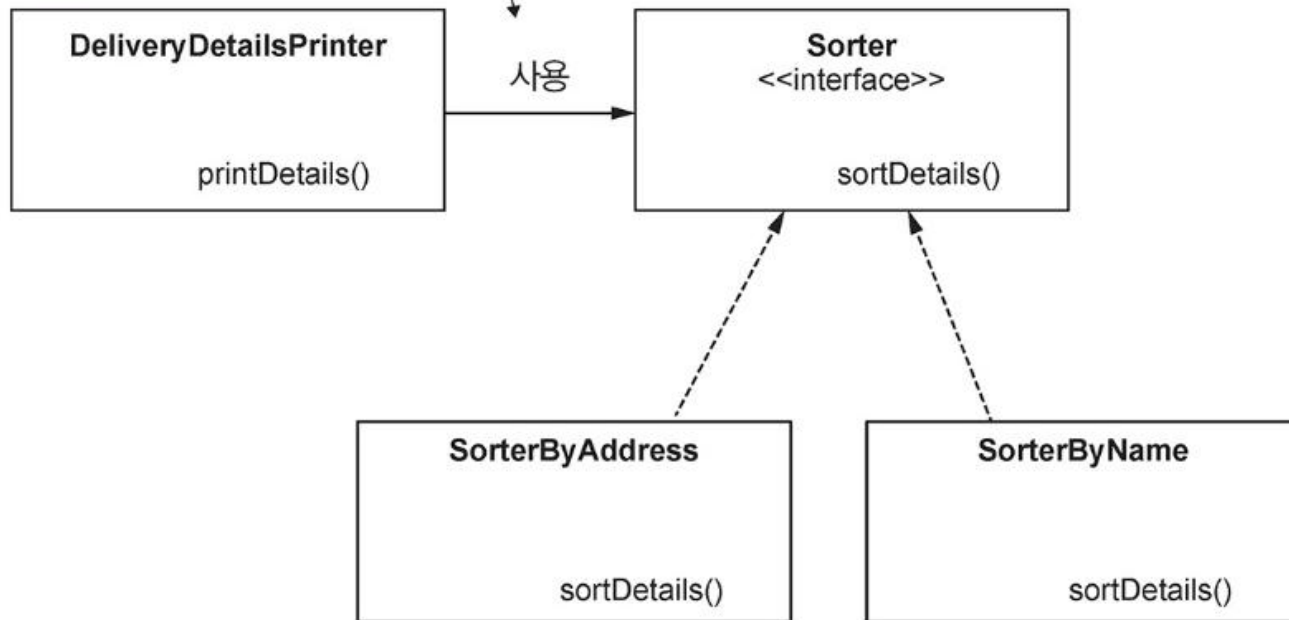
```
    public DeliveryDetailsPrinter(SorterByAddress sorter) {  
        this.sorter = sorter;  
    }
```

```
    public void printDetails() {  
        sorter.sortDetails();  
        // 배송 세부 정보 출력  
    }
```

```
- }
```

정렬 책임을 변경해야 한다면 코드를
변경해야 할 위치는 바로 여기다.

DeliveryDetailsPrinter 객체는
해당 책임을 구현하는 데 필요한 것만 지정한다.
구현에 의존하는 대신 DeliveryDetailsPrinter
객체는 이제 인터페이스에 의존한다.



Sorter 인터페이스는
필요한 것을 정의한다.

이제 동일한 인터페이스를 구현하는 객체를 더 많이 가질 수 있다.
구현에 직접 의존하는 대신 DeliveryDetailsPrinter 객체는 인터페이스(계약)에
의존한다. DeliveryDetailsPrinter는 특정 구현에 종속되지 않고
이 인터페이스를 구현하는 모든 객체를 사용할 수 있다.

✔ Sort 인터페이스 정의

```
public interface Sorter {  
    void sortDetails();  
}
```

```
public class DeliveryDetailsPrinter {  
  
    private Sorter sorter;  
  
    public DeliveryDetailsPrinter(Sorter sorter) {  
        this.sorter = sorter;  
    }  
  
    public void printDetails() {  
        sorter.sortDetails();  
        // 배송 세부 정보 출력  
    }  
}
```

이제 Sorter 인터페이스의 어떤 구현체라도 사용할 수 있으며 더 이상 해당 책임을 사용하는 객체를 변경할 필요가 없다.

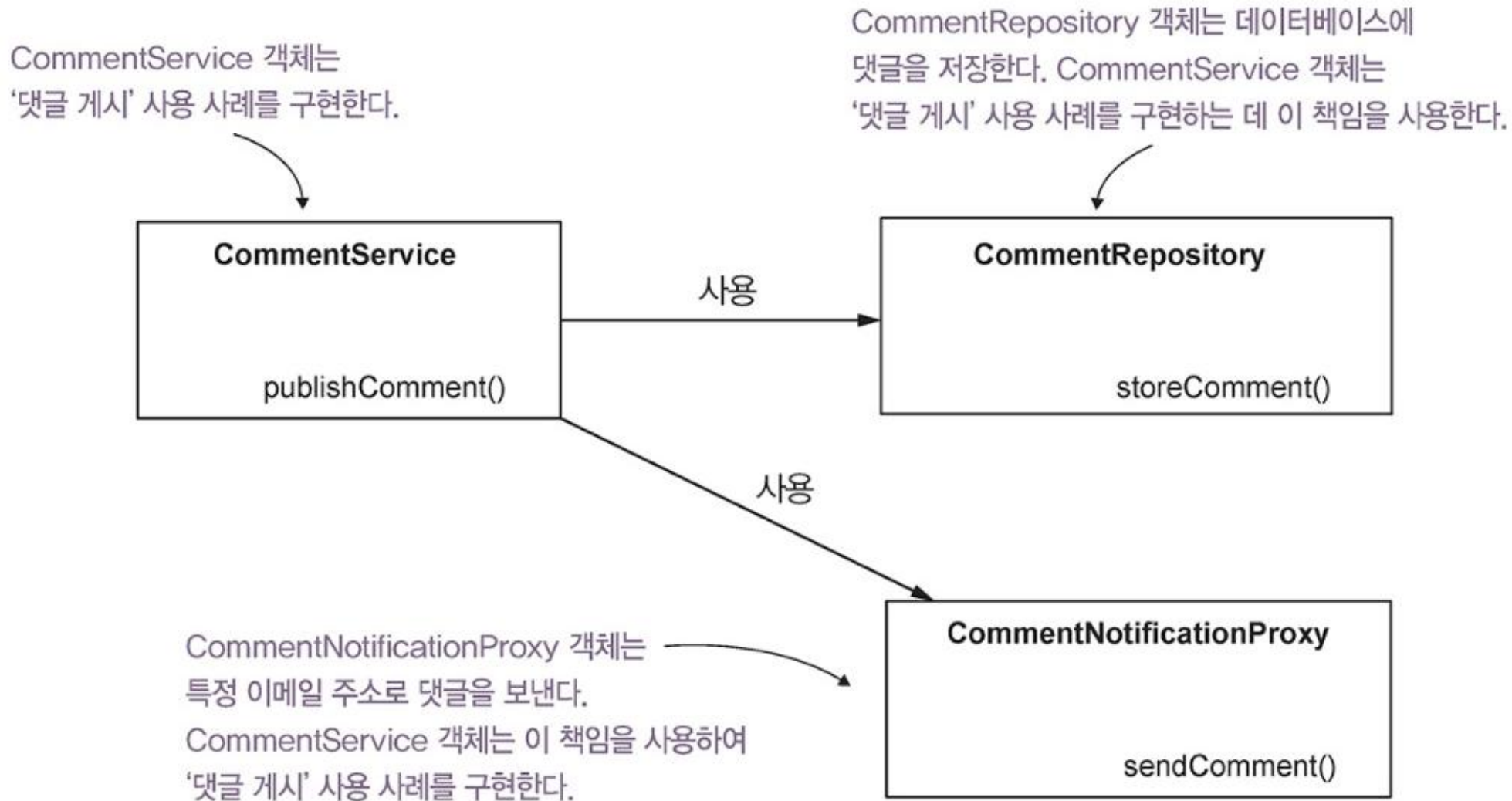
✓ 시나리오 요구 사항

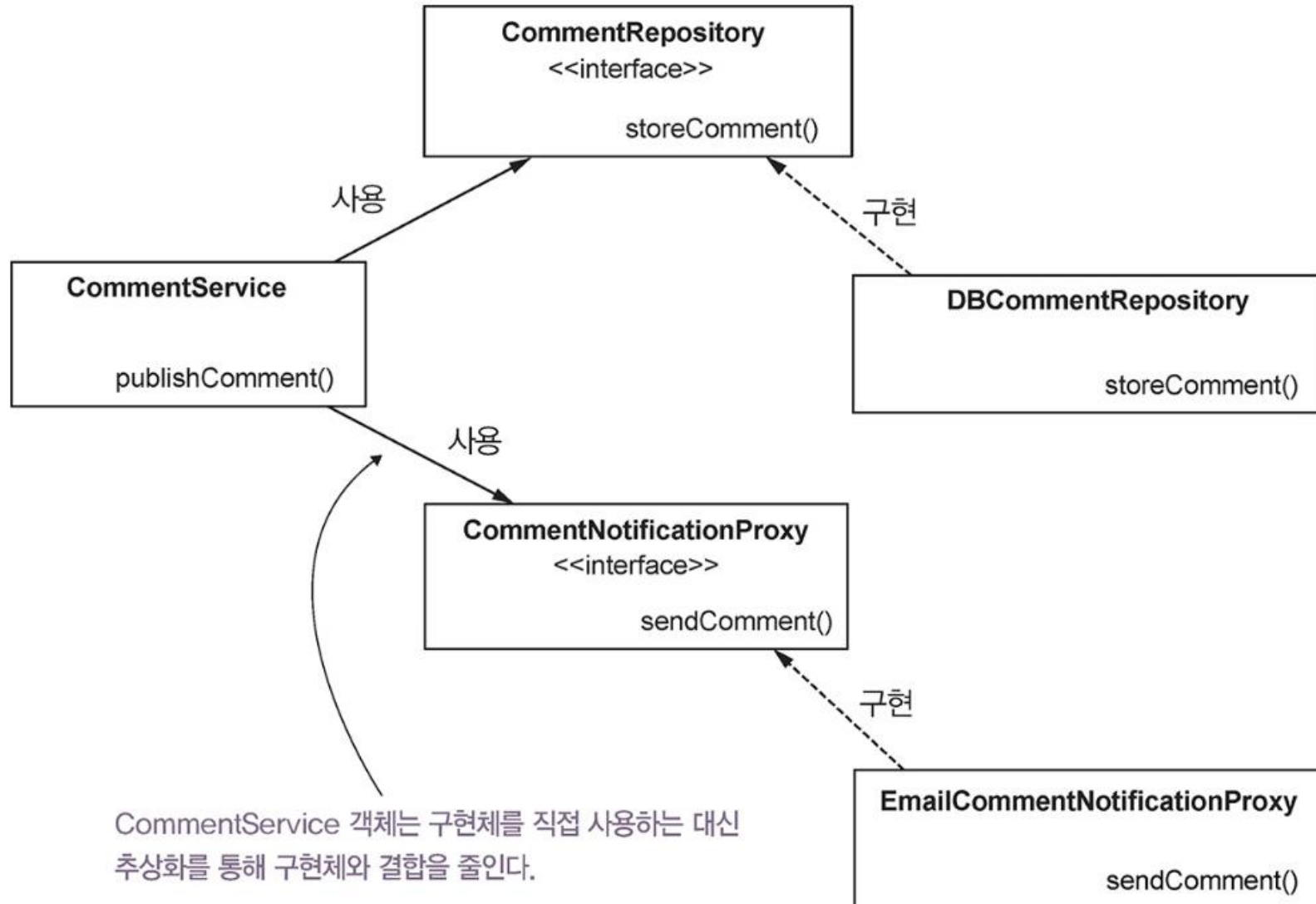
○ 팀 업무 관리용 앱

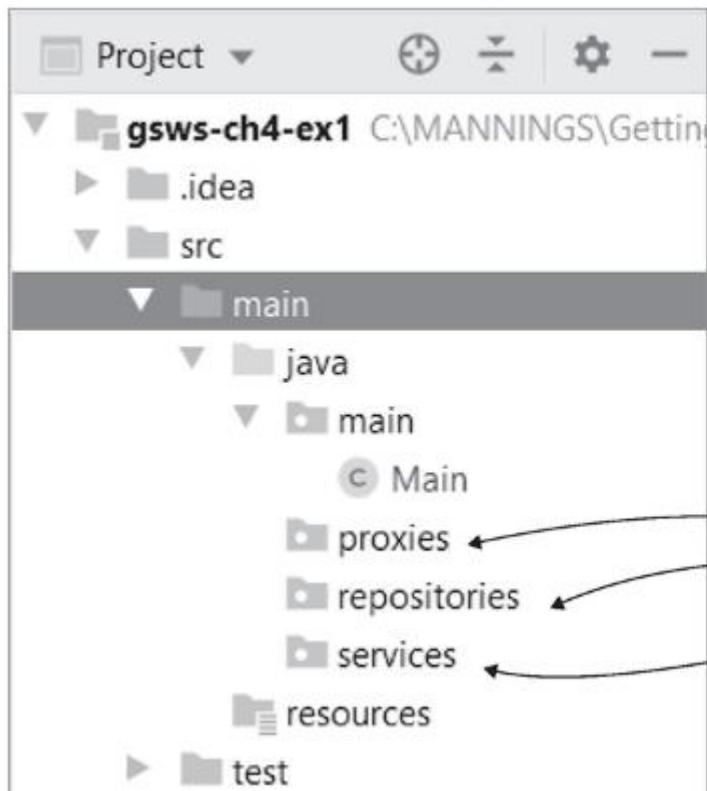
- 사용자가 업무에 대한 댓글을 남길 수 있도록 함
- 사용자가 댓글을 게시하면 해당 댓글은 데이터베이스 등 어딘가에 저장
- 앱은 설정된 특정 주소로 이메일을 보냄

→ 객체를 설계하고 올바른 책임과 추상화를 찾아야 함

✓ 프레임워크 없이 요구 사항 구현







프로젝트 구조를 이해하기 쉽도록
분리된 책임을 각 패키지에 구성한다.

model.Comment.java

```
public class Comment {  
  
    private String author;  
    private String text;  
  
    // getter, setter 생략  
}
```

repositories.CommentRepository.java

```
public interface CommentRepository {  
    void storeComment(Comment comment);  
}
```

repositories.DBCommentRepository.java

```
public class DBCommentRepository implements CommentRepository {  
    @Override  
    public void storeComment(Comment comment) {  
        System.out.println("Storing comment: " + comment.getText());  
    }  
}
```

proxies.CommentNotificationProxy.java

```
public interface CommentNotificationProxy {  
  
    void sendComment(Comment comment);  
}
```

proxies.EmailCommentNotificationProxy.java

```
public class EmailCommentNotificationProxy implements CommentNotificationProxy {  
  
    @Override  
    public void sendComment(Comment comment) {  
        System.out.println("Sending notification for comment: " + comment.getText());  
    }  
}
```

services.CommentService.java

```
public class CommentService {  
  
    private final CommentRepository commentRepository;  
  
    private final CommentNotificationProxy commentNotificationProxy;  
  
    public CommentService(CommentRepository commentRepository, // 생성자 주입으로 의존 객체 설정  
                          CommentNotificationProxy commentNotificationProxy) {  
        this.commentRepository = commentRepository;  
        this.commentNotificationProxy = commentNotificationProxy;  
    }  
  
    public void publishComment(Comment comment) {  
        commentRepository.storeComment(comment);  
        commentNotificationProxy.sendComment(comment);  
    }  
}
```

main.Main.java

```
public class Main {  
  
    public static void main(String[] args) {  
        var commentRepository = new DBCommentRepository();  
        var commentNotificationProxy = new EmailCommentNotificationProxy();  
  
        var commentService = new CommentService(commentRepository, commentNotificationProxy);  
  
        var comment = new Comment();  
        comment.setAuthor("Laurentiu");  
        comment.setText("Demo comment");  
  
        commentService.publishComment(comment);  
    }  
}
```